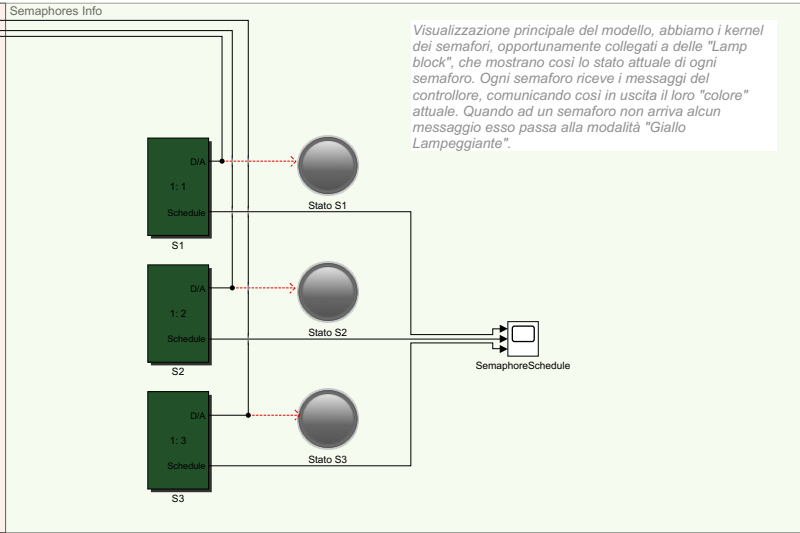
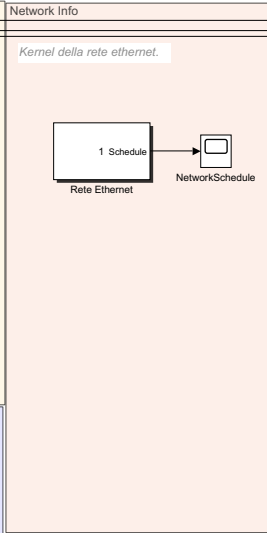
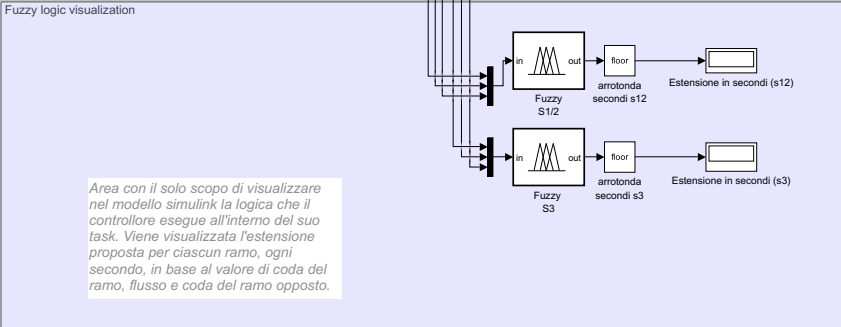
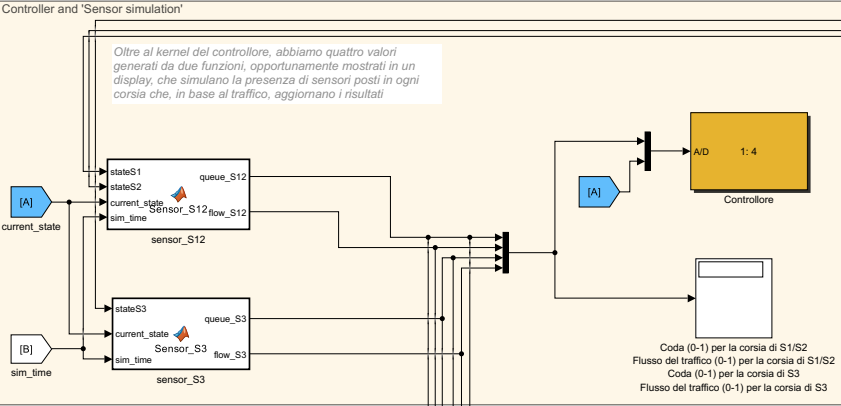


Ora	Secondi dall'inizio
00:00	0
01:00	3600
02:00	7200
03:00	10800
04:00	14400
05:00	18000
06:00	21600
07:00	25200
08:00	28800
09:00	32400
10:00	36000
11:00	39600
12:00	43200
13:00	46800
14:00	50400
15:00	54000
16:00	57600
17:00	61200
18:00	64800
19:00	68400
20:00	72000
21:00	75600
22:00	79200
23:00	82800
24:00	86400

```
function [hour, minute, second] = time_to_hhmmss(time)
% time in hh:mm:ss

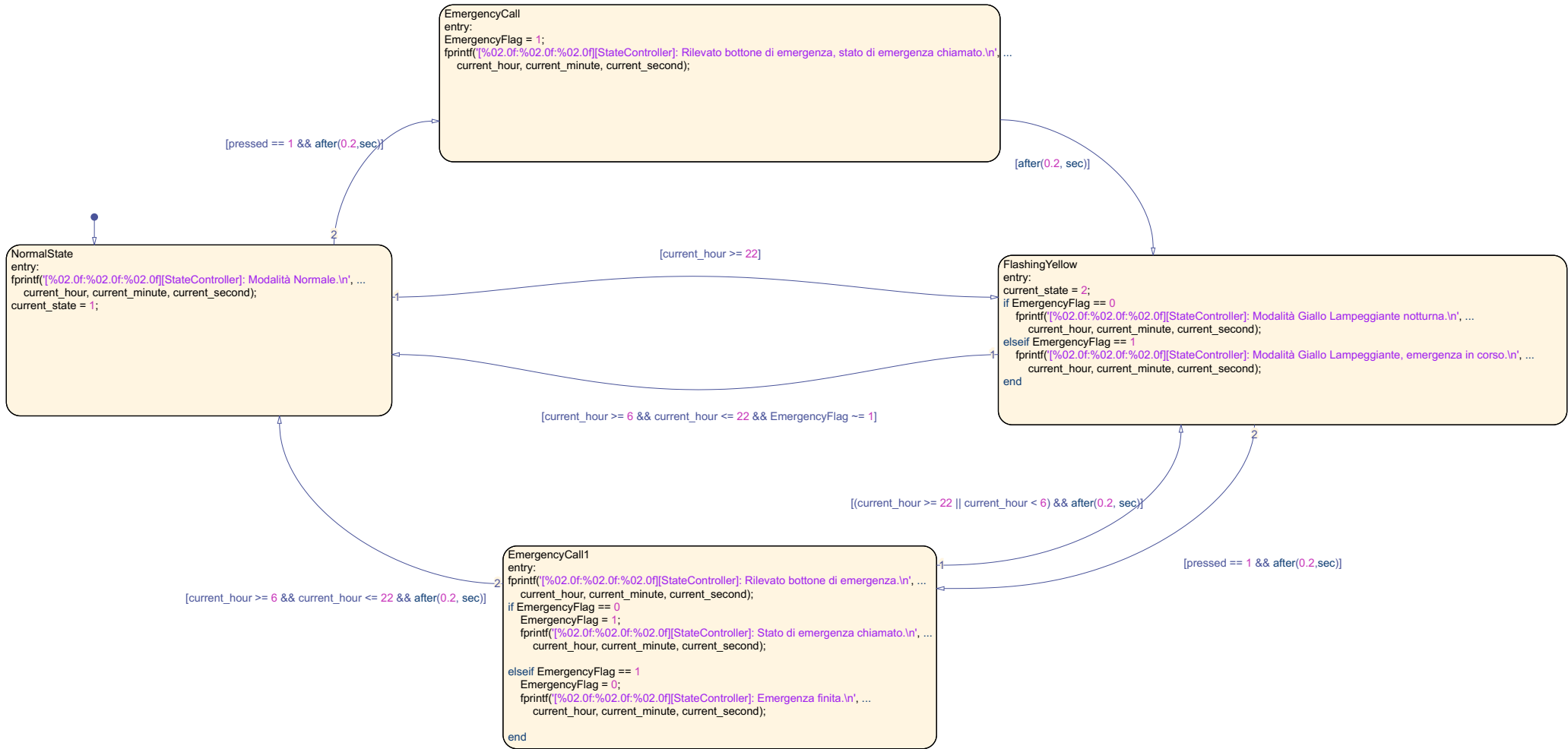
t = mod(time, 24*3600);

hour    = floor(t/3600);           % 0..23
minute  = floor(mod(t,3600)/60);   % 0..59
second  = floor(mod(t,60));        % 0..59
end
```

```
function time_rel = sim_time(time_abs)
% time_abs : tempo assoluto di simulazione (da clock), in secondi
% time_rel : secondi trascorsi dall'inizio simulazione (1,2,3,...)

persistent tStart
if isempty(tStart)
    tStart = time_abs;    % istante di start (es. 43200)
end

elapsed = time_abs - tStart;    % può avere decimali
elapsed = floor(elapsed);      % 0,1,2,... interi
time_rel = elapsed + 1;        % parte da 1 invece che da 0
```



```

function [queue_S12, flow_S12] = Sensor_S12(stateS1, stateS2, current_state, sim_time)
% Sensor_S12 - Sensore aggregato per il ramo S1/S2 (4 corsie)
% stateS1, stateS2: 1=rosso, 2=giallo, 3=verde
% current_state: 1 = normale, 2 = giallo lampeggiante
% queue_S12: coda normalizzata [0,1]
% flow_S12 : flusso medio normalizzato [0,1]
% sim_time: normalizza l'uscita del Clock

```

```

% Parametri
% =====
SEED_S12 = uint32(5);      % per simulazioni diverse

queue_max = 60;           % coda piena = 60 veicoli (4 corsie)
C_max      = 3;           % max veicoli che possono uscire in 1 s (verde)
flow_max   = 2.5;         % flusso medio massimo atteso (veicoli/s)
Nwin       = 5;           % finestra per media flusso (5 s)
% =====

```

```

% Stato interno tra un passo e l'altro
persistent queue arrivalsWindow
if isempty(queue)
    queue = 25;                % coda iniziale
end
if isempty(arrivalsWindow)
    arrivalsWindow = zeros(Nwin,1); % finestra iniziale di zeri
end

```

```

% RNG locale con seed (codegen-friendly)
% =====
persistent rngState seedUsed lastSimTime

```

```

% Inizializza SEMPRE le persistent prima di qualunque uso
if isempty(rngState)
    rngState = uint32(1);
end
if isempty(seedUsed)
    seedUsed = uint32(0);
end
if isempty(lastSimTime)
    lastSimTime = -inf;
end

```

```

newRun = (sim_time < lastSimTime);
lastSimTime = sim_time;

```

```

if newRun || (seedUsed ~= SEED_S12)
    rngState = SEED_S12;
    if rngState == 0
        rngState = uint32(1);
    end
    seedUsed = SEED_S12;
end
% =====

```

```

% profilo orario

```

```

hour_of_day = floor(mod(sim_time/3600, 24)); % 0..23
minute_of_day = floor(mod(sim_time,3600)/60); % 0..59
second_of_day = floor(mod(sim_time,60)); % 0..59

```

```

% Genera arrivi casuali per corsia (4 corsie)

```

```

% =====
if (hour_of_day >= 7 && hour_of_day < 9)
    p_arr_S12 = 0.30;
elseif (hour_of_day >= 9 && hour_of_day < 12)
    p_arr_S12 = 0.26;
elseif (hour_of_day >= 12 && hour_of_day < 14)
    p_arr_S12 = 0.33;
elseif (hour_of_day >= 18 && hour_of_day < 20)
    p_arr_S12 = 0.30;
elseif (hour_of_day >= 22 || hour_of_day < 6)
    p_arr_S12 = 0.02;
else
    p_arr_S12 = 0.22;
end

```

```

% arrivi = sum(rand(4,1) < p_arr_S12); % (linea senza uso di SEED) @@@

```

```

arrivi = 0;
for k = 1:4
    [rngState, r] = randu_xorshift32(rngState);
    if r < p_arr_S12
        arrivi = arrivi + 1;
    end
end

```

```

% =====

```

```

% Verifica se S1/S2 hanno il verde

```

```

hasGreen = (stateS1 == 3) || (stateS2 == 3);

```

```

% Timer verde - conta i secondi consecutivi di verde

```

```

persistent greenTimer
if isempty(greenTimer), greenTimer = 0; end

```

```

% Calcola quante auto escono

```

```

% =====

```

```

if hasGreen
    greenTimer = greenTimer + 1;
else
    greenTimer = 0;
end

```

```

if current_state == 1

```

```

    if hasGreen
        uscita_teorica = min(C_max, queue);
        [rngState, r] = randu_xorshift32(rngState);
        if greenTimer > 20
            fattore = 0.75 + 0.25*r; % 0.75-1.00, media 0.875
        else
            fattore = 0.55 + 0.45*r; % 0.55-1.00, media 0.775
        end

```

```

        uscita = min(queue, round(fattore * uscita_teorica));
    else
        uscita = 0;
    end
end

```

```

end

elseif current_state == 2
    uscita_teorica = min(C_max, queue);
    % fattore = 0.3 + 0.4*rand(); % (linea senza uso di SEED) @@@
    [rngState, r] = randu_xorshift32(rngState);
    fattore = 0.3 + 0.4*r; % 30-70% del max
    uscita = min(queue, round(fattore * uscita_teorica));
else
    uscita = 0;
end
% =====

% Aggiorna coda
queue = queue + arrivi - uscita;
if queue < 0, queue = 0; end
if queue > 80, queue = 80; end

% Aggiorna finestra arrivi per stima flusso medio
arrivalsWindow = [arrivalsWindow(2:end); arrivi];
flowMedia = mean(arrivalsWindow); % veicoli/s medi sugli ultimi Nwin secondi

% Normalizza per dare in uscita [0,1]
% =====
queue_S12 = queue / queue_max;
if queue_S12 > 1
    queue_S12 = 1;
end

%-----@DEBUG@-----
fprintf('%02.0f:%02.0f:%02.0f'[sensor_S12]: S1 queue = %.0f veicoli (norm = %.2f)\n', ...
    hour_of_day, minute_of_day, second_of_day, queue, queue_S12);
%-----

flow_S12 = flowMedia / flow_max;
if flow_S12 > 1
    flow_S12 = 1;
end
% =====

end

function [state, u] = randu_xorshift32(state)
% RNG uniforme [0,1) con stato uint32 (xorshift32)
state = bitxor(state, bitshift(state, 13));
state = bitxor(state, bitshift(state, -17));
state = bitxor(state, bitshift(state, 5));
u = double(state) * 2.3283064365386963e-10; % 1/2^32
end

```

```
function [queue_S3, flow_S3] = Sensor_S3(stateS3, current_state, sim_time)
% Sensor_S3 - Sensore aggregato per il ramo S3 (2 corsie)
% stateS3: 1=rosso, 2=giallo, 3=verde
% current_state: 1 = normale, 2 = giallo lampeggiante
% queue_S3: coda normalizzata [0,1]
% flow_S3 : flusso medio normalizzato [0,1]
% sim_time: tempo simulazione in secondi (uscita del Clock)
```

```
% Parametri
% =====
SEED_S3 = uint32(6);    % simulazioni diverse

queue_max = 40;        % coda piena = 40 veicoli (2 corsie)
C_max      = 2;        % max veicoli che possono uscire in 1 s (verde)
flow_max   = 1.2;      % flusso max realistico (2 corsie, picco ~1.1 veic/s)
Nwin       = 5;        % finestra per media flusso (5 s)
% =====
```

```
% Stato interno tra un passo e l'altro
persistent queue arrivalsWindow
if isempty(queue)
    queue = 10;
end
if isempty(arrivalsWindow)
    arrivalsWindow = zeros(Nwin,1);
end
```

```
% RNG locale con seed (codegen-friendly)
% =====
persistent rngState seedUsed lastSimTime
```

```
% Inizializza SEMPRE le persistent prima di qualunque uso
if isempty(rngState)
    rngState = uint32(1);
end
if isempty(seedUsed)
    seedUsed = uint32(0);
end
if isempty(lastSimTime)
    lastSimTime = -inf;
end
```

```
newRun = (sim_time < lastSimTime);
lastSimTime = sim_time;
```

```
if newRun || (seedUsed ~= SEED_S3)
    rngState = SEED_S3;
    if rngState == 0
        rngState = uint32(1);    % evita stato nullo
    end
    seedUsed = SEED_S3;
end
% =====
```

```
% --- profilo orario ---
hour_of_day = floor(mod(sim_time/3600, 24));    % 0..23
```



```

minute_of_day = floor(mod(sim_time,3600)/60);          % 0..59
second_of_day = floor(mod(sim_time,60));                % 0..59 %#ok<NASGU>

```

```

% Genera arrivi casuali per corsia (2 corsie)
% S3 ramo secondario
% =====
if (hour_of_day >= 7 && hour_of_day < 9)
    p_arr_S3 = 0.32;
elseif (hour_of_day >= 9 && hour_of_day < 12)
    p_arr_S3 = 0.28;
elseif (hour_of_day >= 12 && hour_of_day < 14)
    p_arr_S3 = 0.35;
elseif (hour_of_day >= 18 && hour_of_day < 20)
    p_arr_S3 = 0.32;
elseif (hour_of_day >= 22 || hour_of_day < 6)
    p_arr_S3 = 0.02;
else
    p_arr_S3 = 0.24;
end

```

```

% arrivi = sum(rand(2,1) < p_arr_S3); % (linea senza uso di SEED) @@@
arrivi = 0;
for k = 1:2
    [rngState, r] = randu_xorshift32(rngState);
    if r < p_arr_S3
        arrivi = arrivi + 1;
    end
end
% =====

```

```

% Semaforo verde?
hasGreen = (stateS3 == 3);

```

```

% Timer verde - conta i secondi consecutivi di verde
persistent greenTimer
if isempty(greenTimer), greenTimer = 0; end

```

```

% Calcola quante auto escono
% =====
if hasGreen
    greenTimer = greenTimer + 1;
else
    greenTimer = 0;
end

```

```

if current_state == 1
    if hasGreen
        uscita_teorica = min(C_max, queue);
        [rngState, r] = randu_xorshift32(rngState);
        if greenTimer > 20
            fattore = 0.70 + 0.30*r; % 0.70-1.00, media 0.850
        else
            fattore = 0.50 + 0.50*r; % 0.50-1.00, media 0.750
        end

        uscita = min(queue, round(fattore * uscita_teorica));
    end
end

```

```

    else
        uscita = 0;
    end

elseif current_state == 2
    uscita_teorica = min(C_max, queue);           % stesso limite massimo
    % fattore = 0.3 + 0.4*rand();                 % (linea senza uso di SEED) @@@
    [rngState, r] = randu_xorshift32(rngState);
    fattore = 0.3 + 0.4*r;                       % 30-70% del max
    uscita = min(queue, round(fattore * uscita_teorica));
else
    uscita = 0;
end
% =====

% Aggiorna coda
queue = queue + arrivi - uscita;
if queue < 0, queue = 0; end
if queue > 60, queue = 60; end

% Aggiorna finestra arrivi per stima flusso
arrivalsWindow = [arrivalsWindow(2:end); arrivi];
flowMedia = mean(arrivalsWindow);

% Normalizza
% =====
queue_S3 = queue / queue_max;
if queue_S3 > 1
    queue_S3 = 1;
end

%-----@DEBUG@-----
fprintf('%02.0f:%02.0f:%02.0f][sensor_S3]: S3 queue = %.0f veicoli (norm = %.2f)\n', ...
    hour_of_day, minute_of_day, second_of_day, queue, queue_S3);
%-----

flow_S3 = flowMedia / flow_max;
if flow_S3 > 1
    flow_S3 = 1;
end
% =====

end

function [state, u] = randu_xorshift32(state)
% RNG uniforme [0,1) con stato uint32 (xorshift32)
state = bitxor(state, bitshift(state, 13));
state = bitxor(state, bitshift(state, -17));
state = bitxor(state, bitshift(state, 5));
u = double(state) * 2.3283064365386963e-10; % 1/2^32
end

```